Transactions, Concurrency, Recovery

Outline

- Transaction Concept
- Transaction State
- Concurrent Executions
- Serializability
- Recoverability
- Implementation of Isolation
- Testing for Serializability.

Transaction Concept

- A **transaction** in a **Database Management System (DBMS)** is a **logical unit of work** that consists of one or more SQL operations (such as **INSERT, UPDATE, DELETE, SELECT**) executed as a single unit to ensure **data consistency, integrity, and reliability**.

Example - Fund transfer - transaction to transfer $50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Account A has Money M units → Transfers 50 units to B → Account B has 50 units more money

Either all these instructions have to be executed or none of them

Fund Transfer – a single transaction

Account A has Money M units → Transfers 50 units to B → Account B has 50 units more money

What happens behind – a series of read and write to Database record

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Database instructions

```
delimiter //
create procedure transfer (amount int, note varchar(50), sending_account
int, receiving_account int)
begin

start transaction;
update accounts as A
set A.amount = A.amount - amount
where A.AccountID = sending_account;

update accounts as A
set A.amount = A.amount + amount
where A.AccountID = receiving_account;

insert into transfers values
(TransfersID, amount, sending_account, receiving_account, note, now());
commit work;

end //
delimiter ;
```

Withdraw

Deposit

Money Transaction Table is updated

```
Create table Account
(
AccountID int AUTO_INCREMENT primary key
,amount decimal check (amount >= 0));

Create table Transfers
(
TransfersID int AUTO_INCREMENT primary key
,amount decimal
,from_account_id int not null
,to_account_id int not null
,note varchar(50) not null
,datetime datetime not null
,Constraint from_account_key foreign key (from_account_id) references
Accounts (id)
,Constraint to_account_key foreign key (to_account_id) references Accounts
(id)
);

insert into accounts (id, amount) values (1, (rand()*100));
insert into accounts (id, amount) values (2, (rand()*100));
```

In case of incomplete transactions

```
delimiter //
create procedure transfer (amount int, note varchar(50), sending_account
int, receiving_account int)
begin

start transaction;
update accounts as A
set A.amount = A.amount - amount
where A.AccountID = sending_account;

update accounts as A
set A.amount = A.amount + amount
where A.AccountID = receiving_account;
insert into transfers values
(TransfersID, amount, sending_account, receiving_account, note, now());
commit work;

end //
delimiter ;
```

Had 50,000/-
10,000/- withdrawn

25000/- deposited

What happens if there is a power outage at this point?

Transactions happened – but money transaction table was not updated
Amount = 50000/-

ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
  - That is, for every pair of transactions T_i and T_j, it appears to T_i that either T_j, finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

Example of Fund Transfer

- Transaction to transfer $50 from account A to account B:
  1. read(A)
  2. A := A – 50
  3. write(A)
  4. read(B)
  5. B := B + 50
  6. write(B)
- Atomicity requirement
  - If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
    - Failure could be due to software or hardware
  - The system should ensure that updates of a partially executed transaction are not reflected in the database
- Durability requirement — once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Example of Fund Transfer (Cont.)

- **Consistency requirement** in above example:
  - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
  - Explicitly specified integrity constraints such as primary keys and foreign keys
  - Implicit integrity constraints
    - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand

  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - When the transaction completes successfully the database must be consistent
    - Erroneous transaction logic can lead to inconsistency

Example of Fund Transfer (Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

	T1	T2
1.	**read**(A)	
2.	$A := A - 50$	
3.	**write**(A)	
		read(A), read(B), print(A+B)
4.	**read**(B)	
5.	$B := B + 50$	
6.	**write**(B	

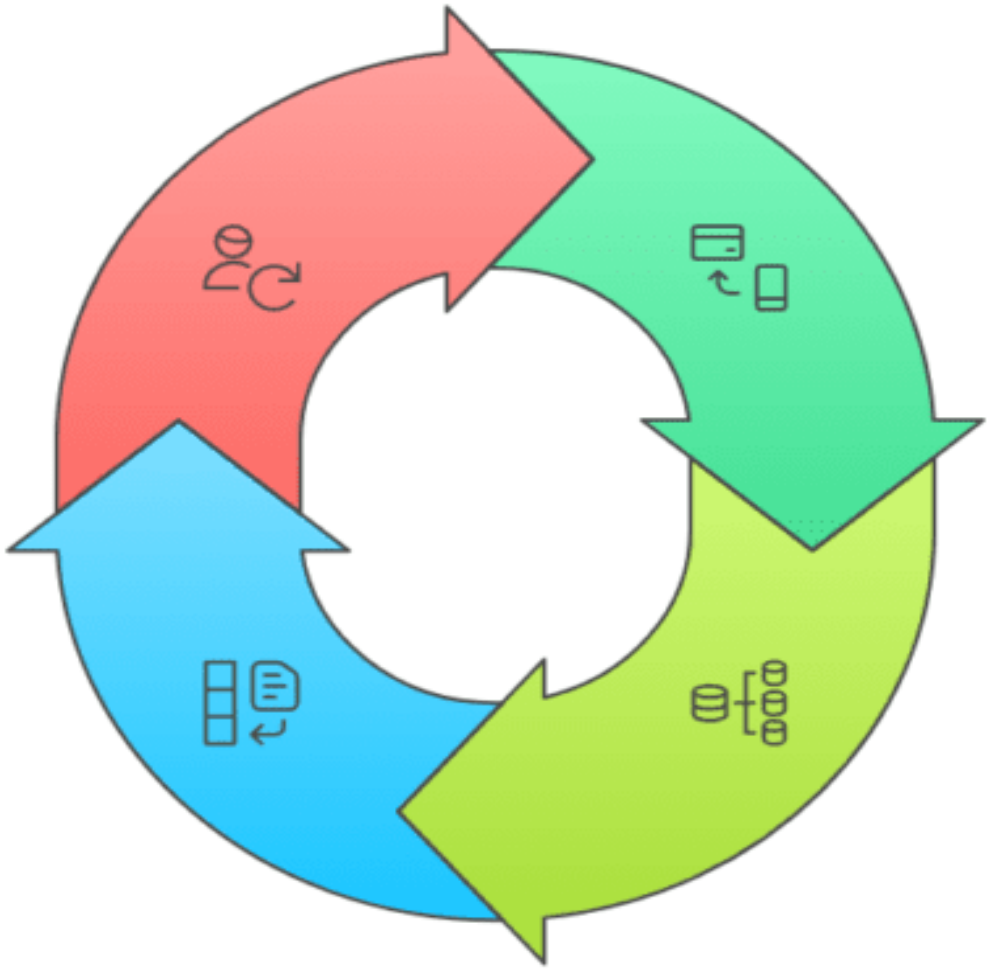
- Isolation can be ensured trivially by running transactions **serially**
  - That is, one after the other.
- ***However, executing multiple transactions concurrently has significant benefits, as we will see later***

Declaring a Transaction in SQL

Transactions are controlled using specific SQL commands.
The most common commands used for managing transactions are:

1. **BEGIN TRANSACTION**: Starts a transaction.
2. **COMMIT**: Commits the transaction, saving all changes made.
3. **ROLLBACK**: Rolls back the transaction, undoing all changes made since the BEGIN TRANSACTION.
4. **SAVEPOINT**: Marks a point within a transaction to which you can later roll back

SQL Transaction Cycle



Begin Transaction

Initiates a new transaction

Execute Operations

Performs database operations

Commit Changes

Saves all changes if successful

Rollback Changes

Reverts changes if an error occurs

Transaction – Bank Transfer

- BEGIN TRANSACTION;
- UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A';
- UPDATE accounts SET balance = balance + 100 WHERE account_id = 'B';
- COMMIT;

Transaction – Cash Withdrawal

- BEGIN TRANSACTION;
- UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A';

- -- Simulate an error (e.g., insufficient balance)
- IF (SELECT balance FROM accounts WHERE account_id = 'A') < 0
- BEGIN
- ROLLBACK; -- Undo changes if balance is negative
- END
- ELSE
- BEGIN
- COMMIT; -- Save changes if balance is valid
- END

Two independent table update

```sql
START TRANSACTION;
UPDATE inventory SET stock = stock - 10 WHERE product_id = 1;
COMMIT; -- This transaction is finalized

START TRANSACTION;
UPDATE orders SET status = 'Shipped' WHERE order_id = 123;
COMMIT; -- This is a separate transaction
```

SAVEPOINT

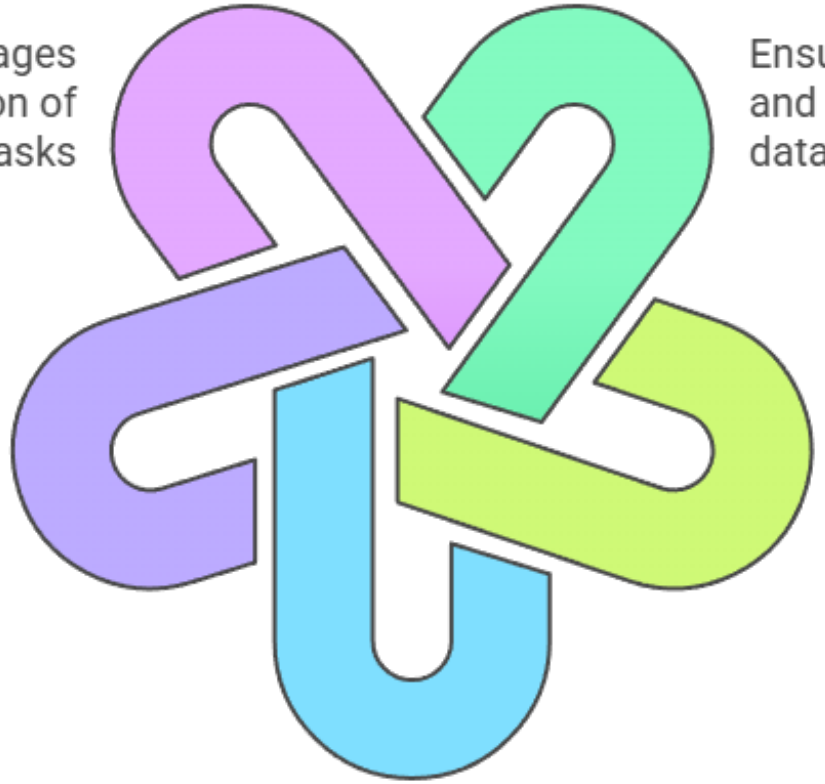
```
BEGIN;
INSERT INTO table1 VALUES(1);
SAVEPOINT my_savepoint;
INSERT INTO table1 VALUES (2);
ROLLBACK TO SAVEPOINT my_savepoint; -- not insert value 2
 INSERT INTO table1 VALUES (3);
COMMIT;
```

```
BEGIN;
INSERT INTO table1 VALUES (3);
SAVEPOINT my_savepoint;
INSERT INTO table1 VALUES (4);
RELEASE SAVEPOINT my_savepoint;
COMMIT;
```

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A';
SAVEPOINT sp1;  -- Marking a safe point

UPDATE accounts SET balance = balance + 100 WHERE account_id = 'B';

ROLLBACK TO sp1;

COMMIT;  -- Commits the transaction only after ensuring everything is correct
```

- If something goes wrong - Rollback to SAVEPOINT instead of rolling back everything and Redo later parts only

Understanding SQL Transactions



Controlled Operations

Manages execution of database tasks

Data Integrity

Ensures accuracy and reliability of data

Prevention of Data Anomalies

Prevents irregularities in data

Consistency

Maintains uniformity across database operations

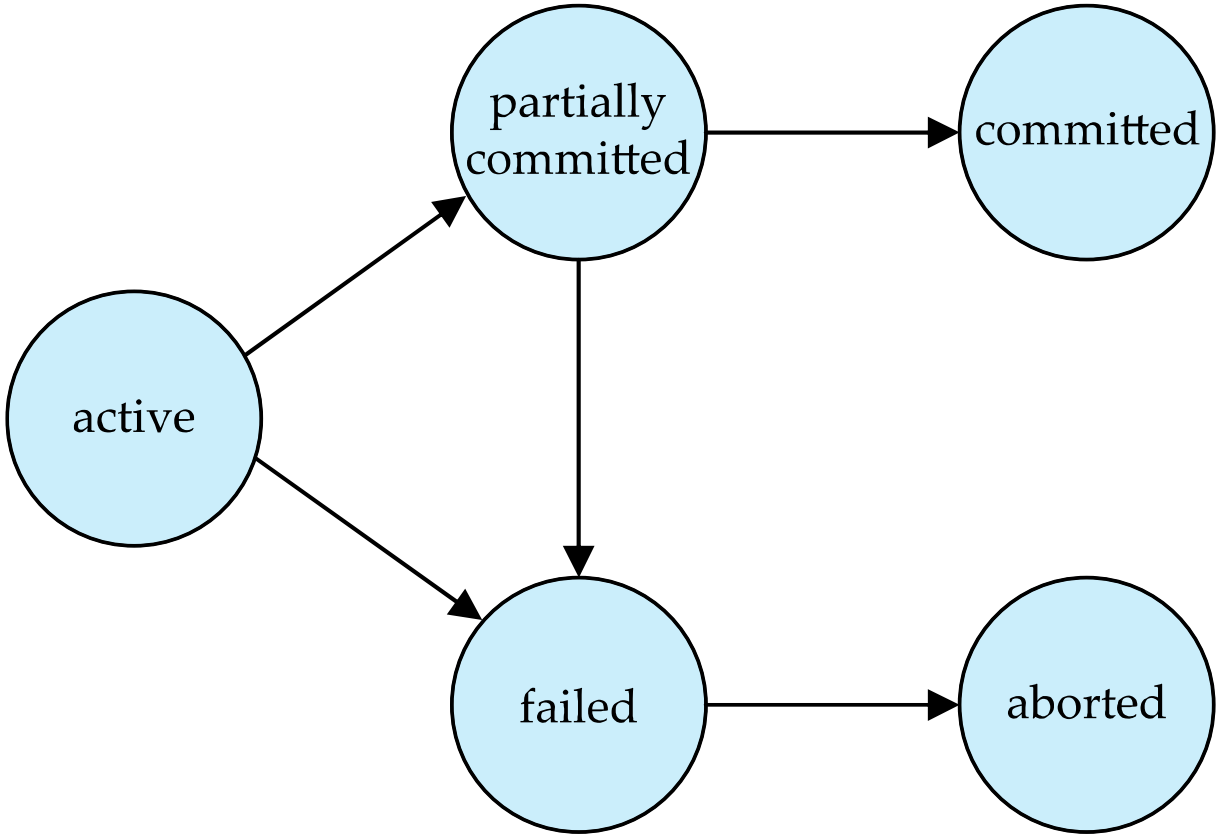
ACID Properties

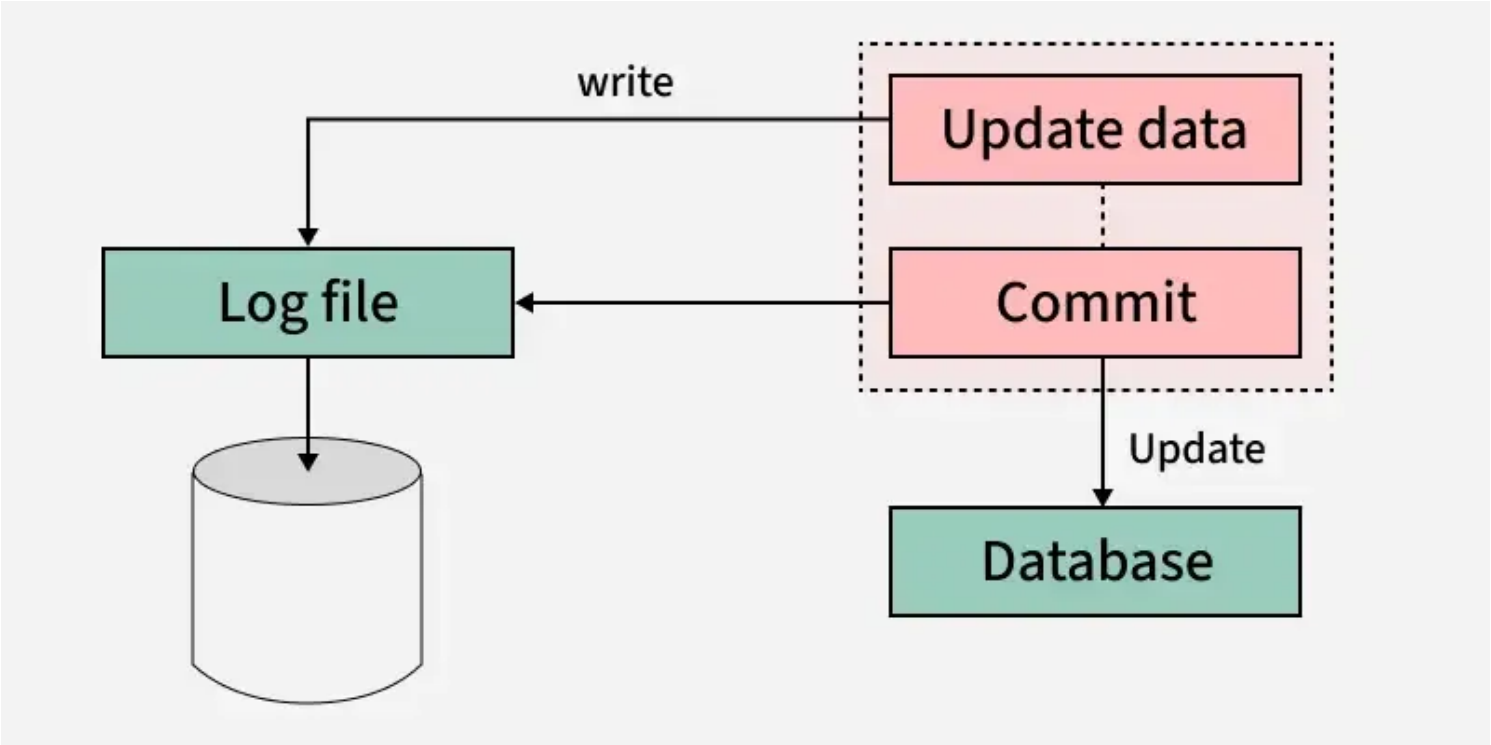
Set of principles for reliable transactions

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction
    - Can be done only if no internal logical error
  - Kill the transaction
- **Committed** – after successful completion.

Transaction State (Cont.)




write
Update data
Log file
Commit
Update
Database

Log in DBMS

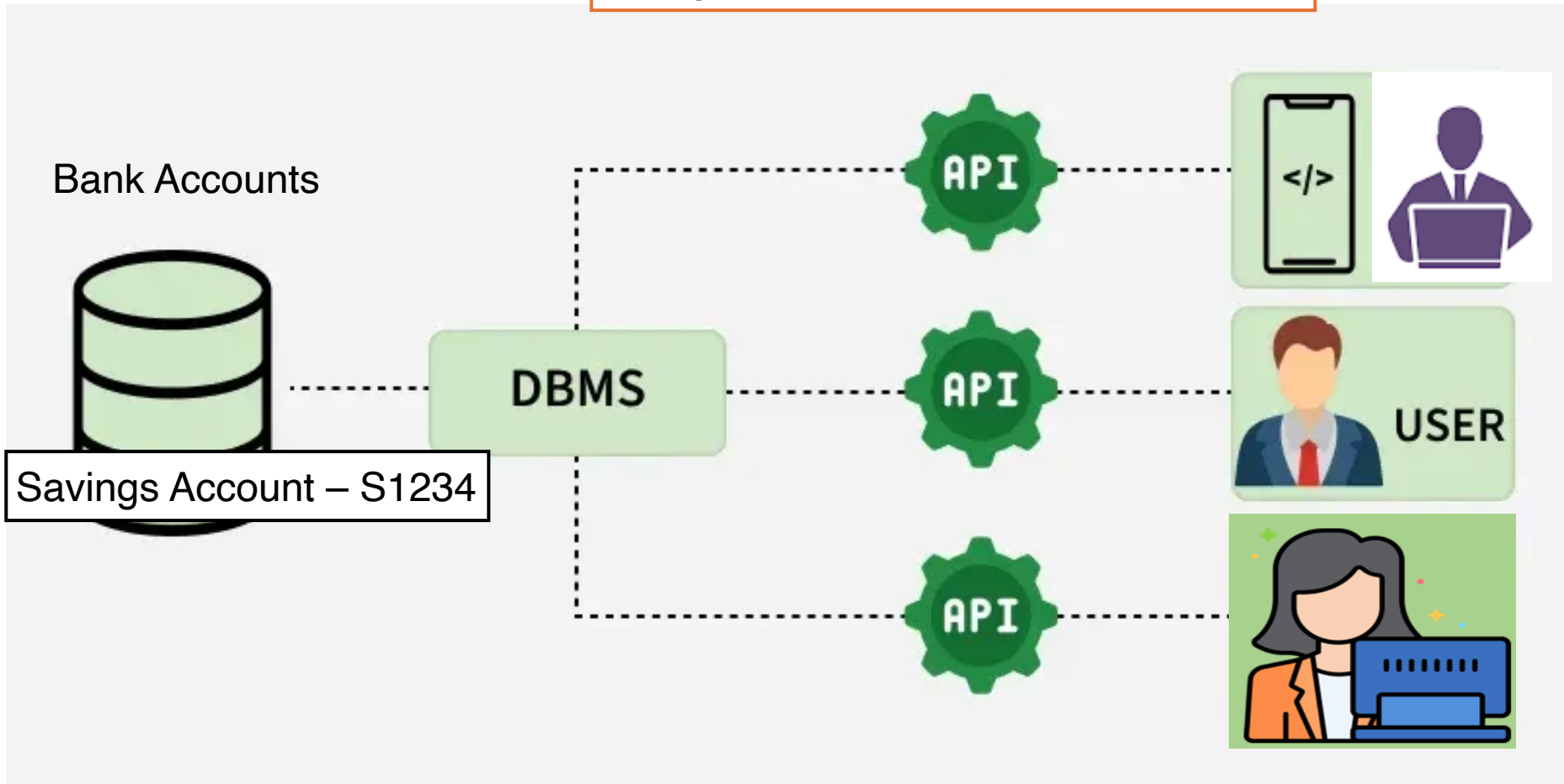
- A log is a sequence of records that document the operations performed during database transactions
- Logs are stored in a log file for each transaction, providing a mechanism to recover data in the event of a failure
- For every operation executed on the database, a corresponding log record is created
- It is critical to store these logs before the actual transaction operations are applied to the database, ensuring data integrity and consistency during recovery processes

```
<T1, Start><T1, Balance, 500, 600><T2, Start><T2,
Balance, 300, 400><T2, Commit><T1, Abort>
```

T1 → Undo
T2 → Redo

Simultaneous access to a Database

All log in at 9:25 am on 1st March, 2026



Deposit some money

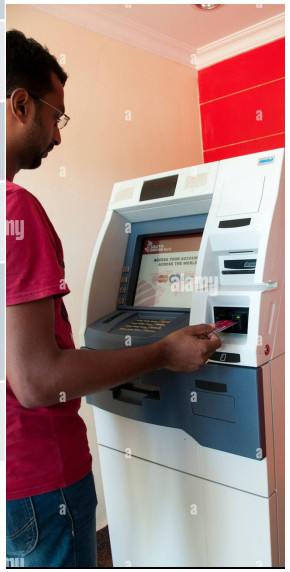
Deposit some money

Withdraw some money

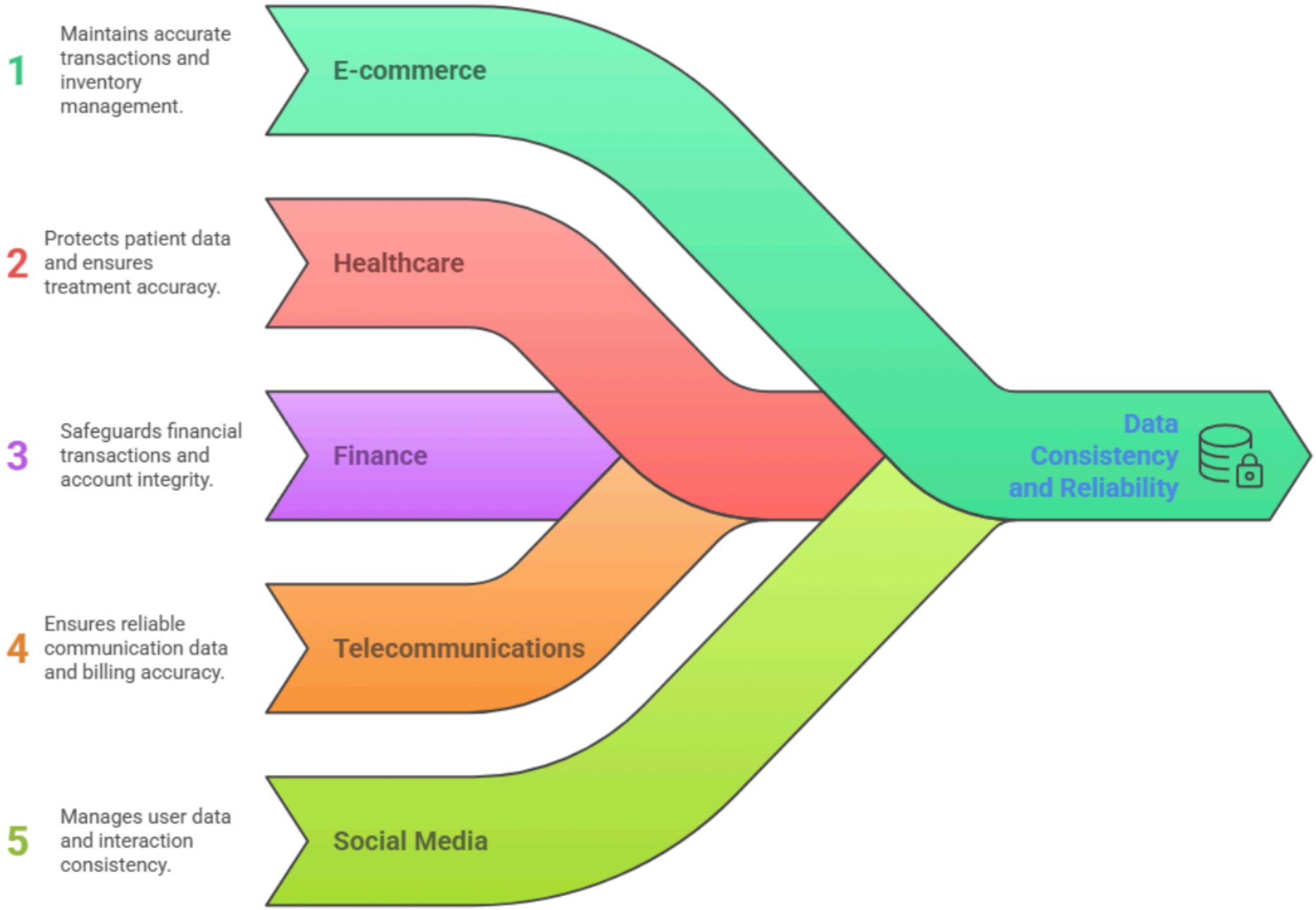
Simultaneous access to DB

A is a joint bank account

T1	T2
Read A = 1000/-	
	Read A = 1000/-
Withdraw 200/- from A	
	Withdraw 300/-
Update A = 800/-	
	Update A = 700/-
Write A	
	Write A



What is the final amount? Is that correct?


1
Maintains accurate transactions and inventory management.
E-commerce
2
Protects patient data and ensures treatment accuracy.
Healthcare
3
Safeguards financial transactions and account integrity.
Finance
4
Ensures reliable communication data and billing accuracy.
Telecommunications
5
Manages user data and interaction consistency.
Social Media
Data Consistency and Reliability

Dirty Read (Reading Uncommitted Data)

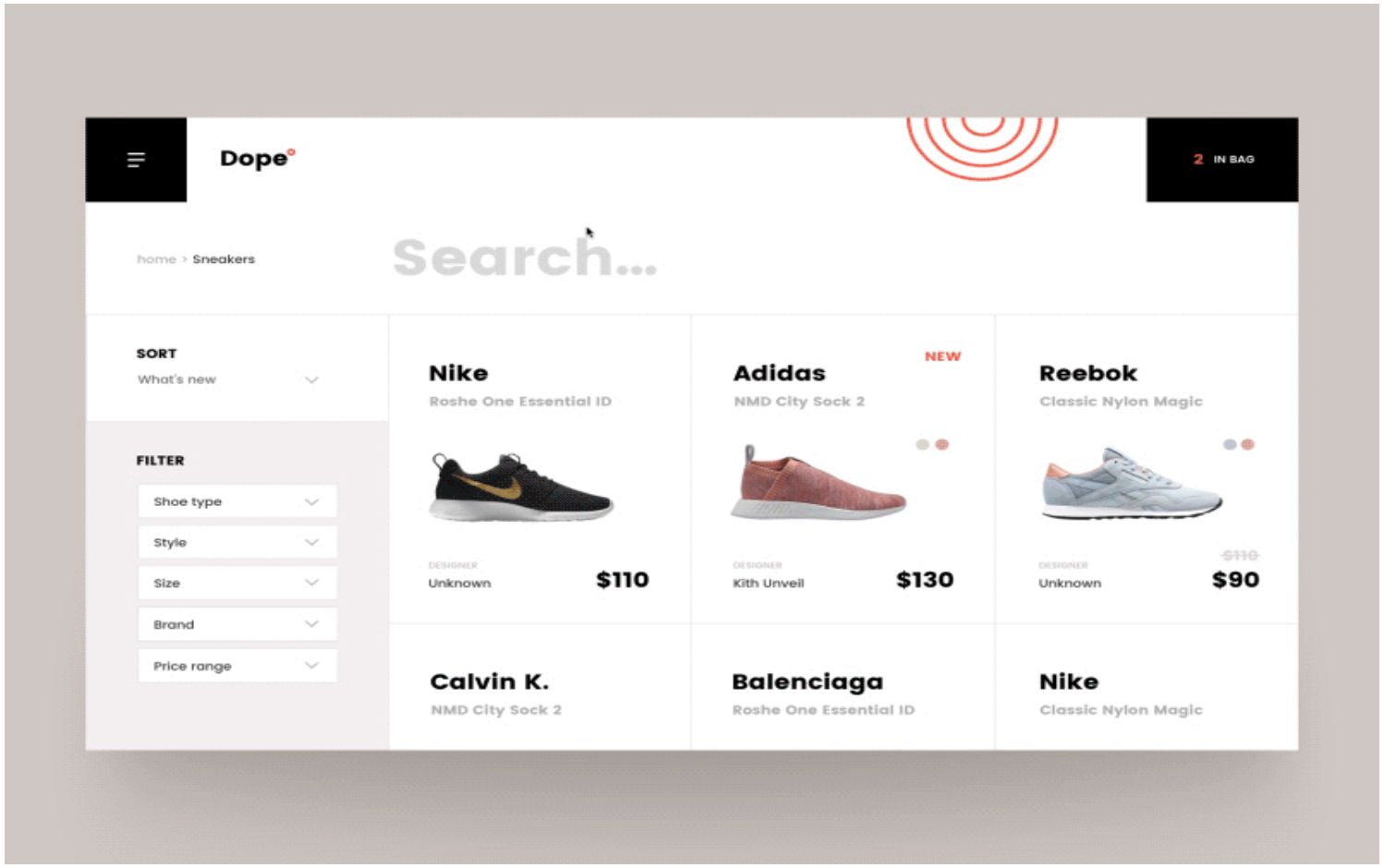
Example Scenario:

- **T1** updates an employee's salary but does **not commit**.
- **T2** reads the updated salary while T1 is still running.
- **T1 rolls back**, but T2 has already used the incorrect salary in calculations.

```sql
BEGIN TRANSACTION;
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 101;

-- Transaction T2: Reads salary before T1 commits
SELECT salary FROM employees WHERE emp_id = 101;  -- Dirty read!

-- T1 rolls back the update
ROLLBACK;
```

Is ACID property always necessary for accessing data?

User-Facing Search and Filter Pages



Non-Repeatable Reads

- **Definition**: A transaction reads **the same data twice** and gets different results because another transaction modified and committed the data between the reads.
- **Example**:
  - Transaction 1 reads a product's stock quantity as 50.
  - Transaction 2 updates the stock to 40 and commits.
  - Transaction 1 reads the stock again and sees 40.
  - **Issue**: The same query within Transaction 1 yields inconsistent results.
- **Impact**: Inconsistent reads can cause errors in applications requiring stable data, but may be allowed for higher throughput

Phantom Reads

- **Definition**: A transaction ***re-executes a query to retrieve a set of rows and finds different rows*** (new or missing) because another transaction inserted or deleted rows and committed.
- **Example**:
  - Transaction 1 queries all orders for a customer, finding 3 orders.
  - Transaction 2 inserts a new order for the same customer and commits.
  - Transaction 1 re-runs the query and finds 4 orders.
  - **Issue**: The result set changes due to “phantom” rows appearing.
- **Impact**: Can affect operations that rely on a consistent set of records, such as generating reports or auditing transactions.

Scenarios

- Allowed
  - Search pages
- Not allowed
  - Preparing reports for a company about how many products have sold in last week

Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
  - **Increased processor and disk utilization**, leading to better transaction *throughput*
    - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
  - **Reduced average response time** for transactions: short transactions need not wait behind long ones.

Concurrency Issues

🔷 Common Concurrency Issues in DBMS

When multiple transactions run at the same time, different types of **anomalies** (inconsistencies) can occur. Here are the three most common issues:

Issue	Cause	Example Effect	Prevented by
Dirty Read	Reading **uncommitted** data	Reading an invalid account balance	**Read Committed** isolation level
Non-Repeatable Read	Reading **committed but changed** data in the same transaction	A product price changes between two reads	**Repeatable Read** isolation level
Phantom Read	Reading a **changing set of rows** due to concurrent inserts/deletes	New orders appearing in a sales report between reads	**Serializable** isolation level

Schedule transactions

- Let T_1 transfer \$50 from A to B, and T_2 transfer 10% of the balance from A to B.
- A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	

Schedule 3

- Let T_1 and T_2 be the transactions defined previously
- The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

- In Schedules 1, 2 and 3, the sum A + B is preserved.

Schedule 4

- This concurrent schedule does not preserve the value of (A + B)

T_1	T_2
read (A)	
A := A – 50	
	read (A)
	temp := A * 0.1
	A := A - *temp*
	write (A)
	read (B)
write (A)	
read (B)	
B := B + 50	
write (B)	
commit	
	B := B + *temp*
	write (B)
	commit

How does a DBMS deal with different types of concurrencies

- Constraints
- Indexes
- Logging and crash recovery
- Locking or MVCC on writes

- Lower isolation levels let transactions see “moving targets.”
- But commits are still checked and applied atomically.

- High isolation (SERIALIZABLE) where needed
  - Explicit row/range locks (e.g., FOR UPDATE)

- Schema design and constraints that reduce dependence on fragile queries
  - Applications choose isolation levels based on importance of read stability.
- For dashboards and searches → weaker isolation is fine.
- For money, inventory, uniqueness constraints → stronger mechanisms or explicit locks

Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.

- Different forms of schedule equivalence give rise to the notions of:
  1. **Conflict serializability**
  2. **View serializability**

Flight Ticket Booking

- Use weaker isolation levels overall (often READ COMMITTED) and
- Enforce the critical “no oversell” rules with locking and constraints, which effectively give conflict-serializable behavior for the specific data involved in seat allocation

- The core seat-allocation logic behaves as if it were ***isolated***
- The rest of the system (search, browsing, reports) runs at weaker isolation and may allow non-repeatable reads and phantoms

- ***Seats(flight_id, seat_no, booked_by, PRIMARY KEY(flight_id, seat_no))***

Booking transaction

- Start transaction (READ COMMITTED is usually enough for this pattern)
- Attempt to INSERT a row for (flight_id, seat_no)
- If insert succeeds → you own that seat
- If insert fails due to PRIMARY KEY or UNIQUE violation → someone else got it first
- Commit

- Uniqueness constraints + row-level locks on index entries ensures correct behavior for seat ownership, even though global isolation level may be lower

Why not full serializable isolation for everything?

- Full SERIALIZABLE isolation (whether conflict-serializable or something stronger like SSI):
- Is expensive in high-throughput, high-contention workloads
- Increases locking, blocking, deadlocks
- Doesn’t add value to less critical operations like:
  - Searching flights
  - Viewing prices
  - Listing your past bookings
- Airline systems need massive concurrency with thousands of agents/users, so they:
- Use lower isolation for read-heavy operations
- Reserve stricter control only for the small critical section that decides:
  - “Is this seat/booking still available?”
  - “Can I safely decrement the available-seat count?”

Concurrency control schemes

Mechanisms to achieve isolation by controlling the interaction among the concurrent transactions in order to prevent them from destroying the desired consistency of the database

Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
  - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

Schedule 1

- Let T_1 transfer \$50 from A to B, and T_2 transfer 10% of the balance from A to B.
- A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	

Schedule 3

- Let T_1 and T_2 be the transactions defined previously
- The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

- In Schedules 1, 2 and 3, the sum A + B is preserved.

Schedule 4

- This concurrent schedule does not preserve the value of (A + B)

T_1	T_2
read (A)	
A := A – 50	
	read (A)
	temp := A * 0.1
	A := A - *temp*
	write (A)
	read (B)
write (A)	
read (B)	
B := B + 50	
write (B)	
commit	
	B := B + *temp*
	write (B)
	commit

Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.

- Different forms of schedule equivalence give rise to the notions of:
  1. **Conflict serializability**
  2. **View serializability**

Simplified view of transactions

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.

Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions wrote Q.
  1. I_i = **read**(Q), I_j = **read**(Q). I_i and I_j don't conflict.
  2. I_i = **read**(Q), I_j = **write**(Q). They conflict.
  3. I_i = **write**(Q), I_j = **read**(Q). They conflict
  4. I_i = **write**(Q), I_j = **write**(Q). They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule

Conflict Serializability

- If a schedule *S* can be transformed into a schedule *S'* by a series of swaps of non-conflicting instructions, we say that *S* and *S'* are **conflict equivalent**

- We say that a schedule *S* is **conflict serializable** if it is ***conflict equivalent*** to a serial schedule

Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule < T_3, T_4 >, or the serial schedule < T_4, T_3 >.

Determine serializability - Construct a Precedence Graph

1. **List all transactions in the schedule**
2. **Find Conflicting Operations:** Two operations **conflict** if they:
    1. Operate on the **same data item**
    2. At least one is a **write**

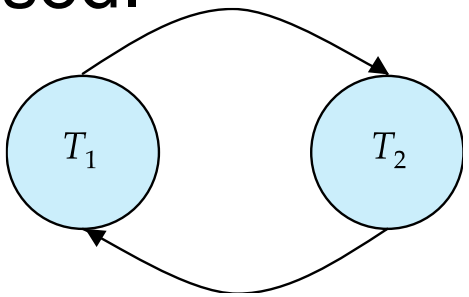
    Read-Write (RW) / Write-Read (WR) / Write-Write (WW)

3. **Add Directed Edges -** If $\mathbf{T_i}$ performs an operation on a data item **before** $\mathbf{T_j}$, and they **conflict**, draw an edge $\mathbf{T_i \rightarrow T_j}$.

4. **Check for Cycles**
    1. If there is **a cycle**, the schedule is **not conflict serializable**.
    2. If there are **no cycles**, the schedule is **conflict serializable**.

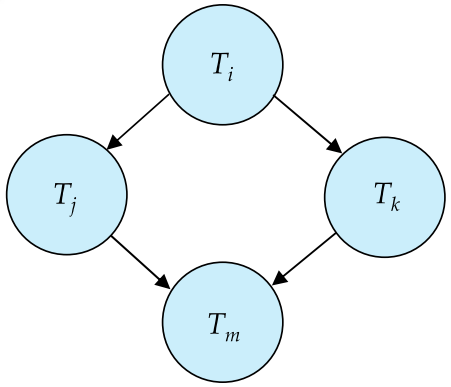
Testing for Serializability

- Consider some schedule of a set of transactions T_1, T_2, ..., T_n
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph



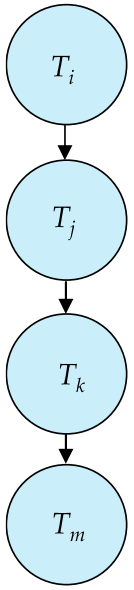
Test for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.

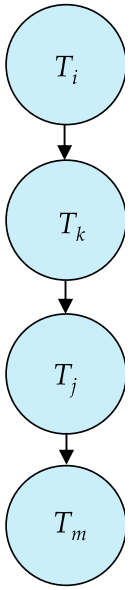


(a)

- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
  - This is a linear order consistent with the partial order of the graph.
  - For example, a serializability order for Schedule A would be $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
    - Are there others?



(b)



(c)

What is View Serializability?

A schedule is **view serializable** if it is **view equivalent** to a **serial schedule**

Two schedules are **view equivalent** if they satisfy the following conditions:

Conditions for View Equivalence:

1. **Same Initial Reads:** If a transaction reads a data item from the database in one schedule, it should read the same initial value in the other schedule.

2. **Same Read-From Relationships:** If a transaction reads a value written by another transaction in one schedule, it should read the same value in the other schedule.

3. **Same Final Writes:** If a transaction is the last one to write a value in one schedule, it should be the last one to write the same value in the other schedule.

Example

```
T1: R(A)
T2:         W(A)
T3:                 R(A)       W(A)
T1:                              W(A)
```

T1 → T2 → T3 → T1

Cycle exists – Not conflict serializable

Using Dependency Graphs to Check View Serializability

- A **dependency graph** can be used to determine if a given schedule is **view serializable – whether final database state would be same perform**

\

```
T1: R(A)
T2:        W(A)
T3:               R(A)      W(A)
T1:                           W(A)
```

Checking View Equivalence: (T1 → T2 → T3→T1)

1. **Same Initial Reads: T1 reads A first**
2. **Same Read-From Relationship: T2 writes A**, and **T3 reads A after T2 writes it**
3. **T3 writes A**, and **T1 writes A at the end**
4. **Same Final Writes:** The last write in both schedules is by T1

Check

```
T1: R(A)   W(A)
T2:          R(A)   W(A)
T3:                    R(A)   W(A)
```

T1 → T2 → T3

- **Conflict serializability is stricter** but simplifies implementation
- **View serializability allows more concurrency** and is common in systems prioritizing availability and performance
- **Databases that relax conflict serializability** use different mechanisms to ensure **optimistic concurrency, or distributed consensus mechanisms**.

Other Notions of Serializability

- The schedule below produces same outcome as the serial schedule < T_1, T_5 >, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
read (A)	
A := A – 50	
write (A)	
	read (B)
	B := B - 10
	write (B)
read (B)	
B := B + 50	
write (B)	
	read (A)
	A := A + 10
	write (A)

- Determining such equivalence requires analysis of operations other than read and write.

Test for View Serializability

- The precedence graph test for conflict serializability cannot be used directly to test for view serializability.
  - Extension to test for view serializability has cost exponential in the size of the precedence graph.
- The problem of checking if a schedule is view serializable falls in the class of *NP*-complete problems.
  - Thus, existence of an efficient algorithm is *extremely* unlikely.
- However practical algorithms that just check some **sufficient conditions** for view serializability can still be used.

Ensuring Serializability

- SQL databases ensure **serializability** — the highest level of transaction isolation—using **locking mechanisms, timestamp ordering, and concurrency control techniques**
- **Isolation Levels in SQL -** control concurrency behavior
  - The highest isolation level—**SERIALIZABLE**—ensures that transactions execute in a manner equivalent to some serial order.

SQL Isolation Levels (from weakest to strongest):

Isolation Level	Prevents Dirty Reads?	Prevents Non-Repeatable Reads?	Prevents Phantom Reads?	Ensures Serializability?
Read Uncommitted	❌ No	❌ No	❌ No	❌ No
Read Committed	✅ Yes	❌ No	❌ No	❌ No
Repeatable Read	✅ Yes	✅ Yes	❌ No	❌ No
Serializable	✅ Yes	✅ Yes	✅ Yes	✅ Yes

```sql
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;

-- Your SQL queries

COMMIT;
```

- **Effect**: Transactions are executed as if they were in a serial order, even if executed concurrently.

- **Method Used**: Typically enforced using **strict two-phase locking (2PL)** or **timestamp ordering**.

Mechanisms to Ensure Serializability

Two-Phase Locking (2PL)

- Every transaction **acquires locks** before reading/writing.
- Once a transaction releases a lock, it **cannot acquire new locks** (hence "two-phase").
- Prevents conflicts that break serializability.

Strict Two-Phase Locking (S2PL)

- A variation where locks are **held until the transaction commits** (ensuring recoverability).

Timestamp Ordering

- Each transaction gets a timestamp when it starts.
- Read and write operations **must respect timestamp order**.
- Transactions violating serializability are aborted

Detecting and Preventing Serializability Violations

- **Conflict detection mechanisms** - checks conflicting reads/writes
- **Deadlock detection** : identifying cycles in precedence graphs
- **Rollback & Retry** : aborting transactions that would cause inconsistency

Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work

Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i, then the commit operation of T_i appears before the commit operation of T_j.
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

Cascadeless Schedules

Definition of Cascadeless Schedules - A schedule is **cascadeless** if **a transaction is only allowed to read committed data** (i.e., it does not read uncommitted changes from other transactions).

Condition for a Cascadeless Schedule

👉 **T2 is not allowed to read a value written by T1 until T1 has committed.**

- This prevents cascading rollbacks because transactions never depend on uncommitted data

Non-Cascadeless Schedule (With Cascading Rollback)

Time	Transaction 1 (T1)	Transaction 2 (T2)
t1	Write(X)	
t2		Read(X) (from T1)
t3	❌ (T1 fails)	❌ (T2 must roll back)

T2 reads X before T1 commits, so if T1 fails, T2 must also roll back → **cascading rollback!** ❌

Cascadeless Schedule (No Cascading Rollback)

Time	Transaction 1 (T1)	Transaction 2 (T2)
t1	Write(X)	
t2	Commit (T1)	
t3		Read(X) (now allowed)

Since **T2 reads X only after T1 commits**, there is no chance of a cascading rollback. ✅

Who Issues COMMIT command?

(A) Application Programs / Users

- When a user writes and executes SQL queries, they **explicitly** issue COMMIT to save changes.

```sql
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 102;
COMMIT;  -- Saves the transaction permanently
```

(B) DBMS Itself (Implicit Commit)

Some SQL commands **automatically trigger a COMMIT** in certain DBMSs.

Examples:

- **DDL Commands** (CREATE, DROP, ALTER)
- **GRANT/REVOKE** (Permissions)
- Some **database configurations** force a commit after each statement

Who issues COMMIT command?

```sql
CREATE TABLE employees (id INT, name VARCHAR(50));
-- This automatically commits any previous transactions
```

3. Transaction Manager (in Multi-Transaction Systems)

•In enterprise systems, **DBMS transaction managers** decide when to commit transactions based on rules or atomicity conditions

•Example: In a **two-phase commit (2PC)** system, the **coordinator** decides when to issue a COMMIT across distributed databases.

DBMS Transaction Manager

A component of a **Database Management System (DBMS)** responsible for **ensuring that database transactions** are executed **safely and consistently**

It **manages transaction execution, concurrency control, and recovery mechanisms** to ensure database integrity.

Transaction Control

- Ensures that transactions follow **ACID properties** (Atomicity, Consistency, Isolation, Durability).
- Begins, commits, and rolls back transactions.

BEGIN TRANSACTION; UPDATE accounts SET balance = balance - 500 WHERE account_id = 101; UPDATE accounts SET balance = balance + 500 WHERE account_id = 102; COMMIT;

Transaction Manager ensures atomic execution

Concurrency Control

Prevents problems like **dirty reads, lost updates, and uncommitted dependencies** in a multi-user environment.

Uses **locking mechanisms** (e.g., two-phase locking), **timestamps**, or **optimistic concurrency control**.

Transaction Scheduling

- Determines the execution order of transactions.
- Ensures transactions are executed **serializably** to maintain database correctness.

Deadlock Detection & Prevention

- **Deadlock** occurs when two or more transactions **wait indefinitely** for resources.
- The **Transaction Manager** detects and resolves deadlocks by aborting or rolling back transactions.

Example

Multi transactions trying to update the same two tables in a different order can lead to a deadlock. The transaction manager detects and breaks this deadlock.

Example:

- **Transaction T1:** Locks **Row A**, then waits for **Row B** (locked by T2).
- **Transaction T2:** Locks **Row B**, then waits for **Row A** (locked by T1).
- DBMS detects this **circular dependency** and rolls back **one transaction**.

Recovery & Rollback

- Ensures **durability** by maintaining logs of committed transactions.
- Uses **undo (rollback) logs** and **redo logs** to recover from failures. If a system crashes before a transaction commits, the **Transaction Manager** rolls back incomplete changes to maintain database consistency.

How Does the Transaction Manager Work?

Receives Transaction Requests → From applications or users

Manages Transaction Execution → Using concurrency control and scheduling

Handles Commits or Rollbacks → Ensures ACID compliance

Logs Changes for Recovery → Maintains logs to handle crashes

Ensures Durability → Guarantees data is permanently saved

Example: online banking system where **User A transfers ₹500 to User B**.

→ **Transaction Starts:**

- Deduct ₹500 from **User A's Account**
- Add ₹500 to **User B's Account**

→ **System Crash Happens Before Completion!**

- Without a **Transaction Manager**, only the deduction might be recorded, leading to **data inconsistency**.

→ **With a Transaction Manager:**

- If the transaction **wasn't committed**, it **rolls back** (User A gets the money back).
- If it **was committed**, it **recovers and completes** the update for User B.

Who issues COMMIT command?

- **Database Triggers & Stored Procedures**
  - If a stored procedure or trigger is programmed to include COMMIT. it will issue the command

```
CREATE TRIGGER after_order_update
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    INSERT INTO audit_log VALUES (NOW(), 'Order updated');
    COMMIT;  -- Saves the log permanently
END;
```

Who issues ROLLBACK COMMAND

User/Application (Manually)

- A **developer or database user** can manually issue a **ROLLB** r transac
- This is rect data fr

```sql
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 102;
ROLLBACK;  -- Cancels all changes and restores the original balance
```

DBMS Transaction Manager (Automatically)

- The **Transaction Manager** automatically issues **ROLLBACK** when:
  - A transaction **violates integrity constraints** (e.g., primary key conflict).
  - A **deadlock** occurs, and one transaction must be aborted.
  - A **system crash** happens before the transaction

```
import mysql.connector

conn = mysql.connector.connect(user='user', password='password', database='bank')
cursor = conn.cursor()

try:
    cursor.execute("UPDATE accounts SET balance = balance - 500 WHERE account_id = 10
    cursor.execute("UPDATE accounts SET balance = balance + 500 WHERE account_id = 10
    conn.commit()  # Commit if no error
except:
    conn.rollback()  # Rollback if an error occurs
```

Trigger or Constraint Violation

- If a da
  CHEC
  autom

```
BEGIN TRANSACTION;
INSERT INTO employees (emp_id, name, department_id) VALUES (101, 'Alice', 5);
-- If department_id = 5 does not exist in the departments table, FOREIGN KEY constrai
-- The DBMS automatically issues a ROLLBACK
```

When is ROLLBACK Issued?

Before a COMMIT is issued

- If a transaction has **not been committed**, it can be **rolled back** to its initial state.

On Transaction Failure

- If a transaction **fails due to an error** (e.g., data type mismatch, integrity constraint violation).

On System Crash or Power Failure

- If the system **crashes before committing a transaction**, the DBMS **automatically rolls back** any uncommitted changes.

When a Deadlock Occurs

- If two transactions are **waiting indefinitely** for resources, the DBMS selects one transaction to **ROLLBACK** and allows the other to proceed.

Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
  - either conflict or view serializable, and
  - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
  - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.

Dirty Reads

- A **dirty read** occurs when a transaction reads data that has been **modified but not yet committed** by another transaction. If the modifying transaction later **rolls back**, the data read by the first transaction becomes **invalid or inconsistent**, leading to potential errors.

Scenario:

- **Transact** committed yet.
- **Transact** ance.
- **T1 is roll** ata.

```
-- Transaction T1: Modify balance (but do not commit yet)
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;

-- Transaction T2: Read the balance while T1 is uncommitted
SELECT balance FROM accounts WHERE account_id = 1;  -- Dirty read!

-- T1 rolls back the transaction (changes are undone)
ROLLBACK;
```

Why Are Dirty Reads Dangerous?

- Transactions might **make decisions based on incorrect data**.
- Leads to **inconsistent database states**.
- Can caus ct fund transfers

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Concurrency Control (Cont.)

- Schedules must be conflict or view serializable, and recoverable, for the sake of database consistency, and preferably cascadeless.
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency.
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur.
- Some schemes allow only conflict-serializable schedules to be generated, while others allow view-serializable schedules that are not conflict-serializable.

Concurrency Control vs. Serializability Tests

- Concurrency-control protocols allow concurrent schedules - but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols (generally) do not examine the precedence graph as it is being created
  - Instead a protocol imposes a discipline that avoids non-serializable schedules.
  - We study such protocols in Chapter 16.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.

Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
  - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
  - E.g., database statistics computed for query optimization can be approximate (why?)
  - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance

Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
  - Repeated reads of same record must return same value.
  - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
  - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.

Levels of Consistency

- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default
- E.g., Oracle (and PostgreSQL prior to version 9) by default support a level of consistency called snapshot isolation (not part of the SQL standard)

Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
  - **Commit work** commits current transaction and begins a new one.
  - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
  - Implicit commit can be turned off by a database directive
    - E.g., in JDBC -- connection.setAutoCommit(false);
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
    - E.g. In SQL **set transaction isolation level serializable**
    - E.g. in JDBC -- connection.setTransactionIsolation(

      Connection.TRANSACTION_SERIALIZABLE)

Implementation of Isolation Levels

- Locking
  - Lock on whole database vs lock on items
  - How long to hold lock?
  - Shared vs exclusive locks
- Timestamps
  - Transaction timestamp assigned e.g. when a transaction begins
  - Data items store two timestamps
    - Read timestamp
    - Write timestamp
  - Timestamps are used to detect out of order accesses
- Multiple versions of each data item
  - Allow transactions to read from a “snapshot” of the database

Transactions as SQL Statements

- E.g., Transaction 1:
  select *ID, name* **from** *instructor* **where** *salary* > 90000
- E.g., Transaction 2:
  insert into *instructor* **values** ('11111', 'James', 'Marketing', 100000)
- Suppose
  - T1 starts, finds tuples salary > 90000 using index and locks them
  - And then T2 executes.
  - Do T1 and T2 conflict? Does tuple level locking detect the conflict?
  - Instance of the **phantom phenomenon**
- Also consider T3 below, with Wu's salary = 90000
  update *instructor*
  set *salary* = *salary* * 1.1
  where *name* = 'Wu'
- Key idea: Detect "**predicate**" conflicts, and use some form of "**predicate locking**"

End of Chapter 17